



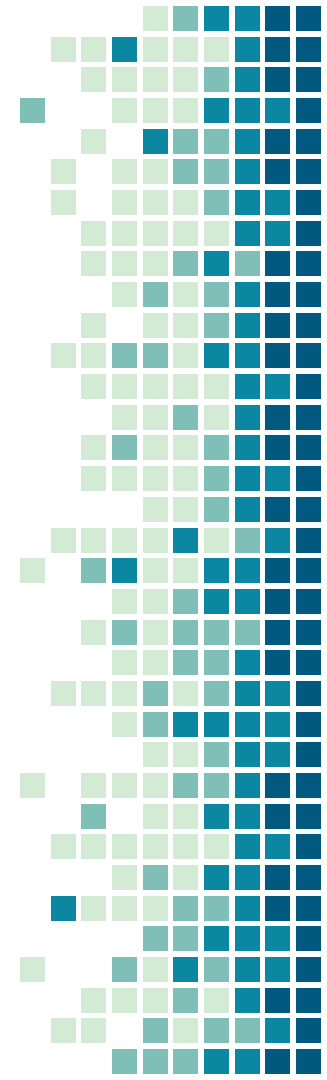
CdL in Informatica - A.A. 2024 - 2025

Programmazione 1 - Modulo 2

Lezione 7
24/10/2024

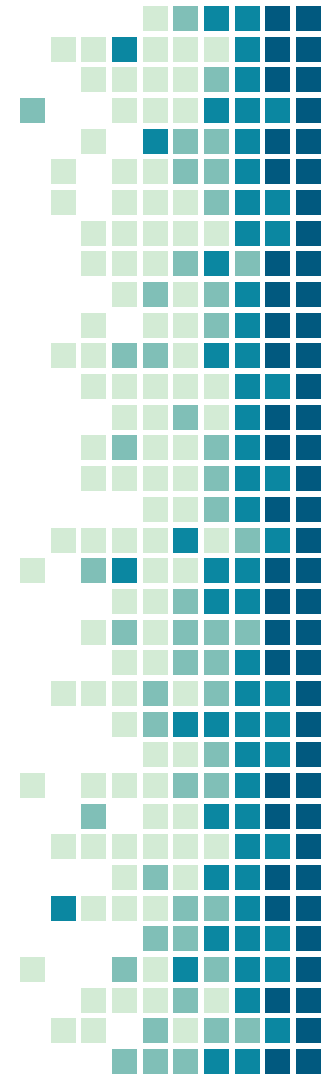
Andrea Loddo

Federico Meloni - Alessandra Perniciano - Fabio Pili



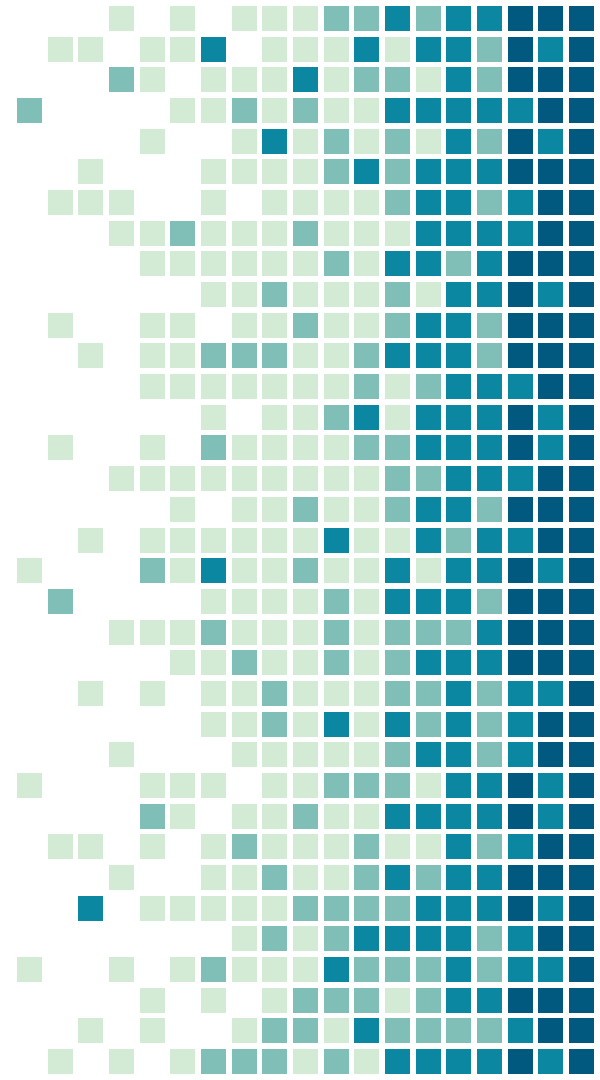
Argomenti

- Array multidimensionali
- Organizzazione della memoria
- Inizializzazione matrici



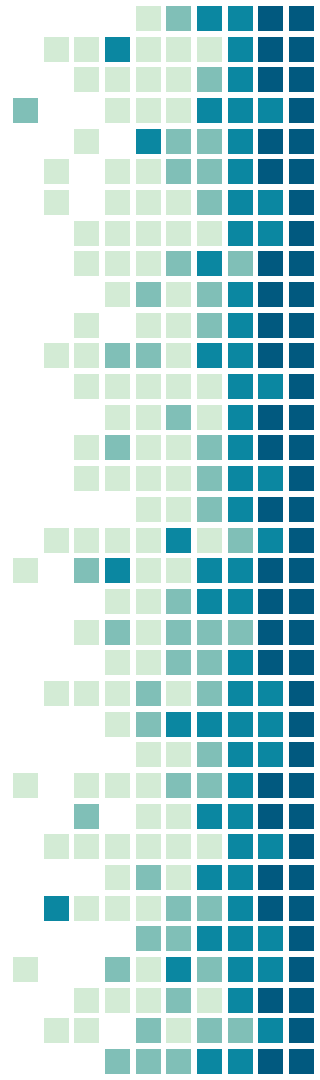
Tipi di dato non primitivi

Gli array multidimensionali



Array multidimensionali

- Esempio: gestione dei voti dell'intero corso di PR1?
 - Un solo esame: array di 150 elementi
- Se volessi gestire 2, 3, 4 esami?
 - 2, 3, 4 array...
- Se ne volessi gestire 30?
 - Posso costruire un array di array?
- Un array multidimensionale sfrutta il meccanismo di funzionamento degli array per rappresentare dati a due (o più) dimensioni
 - Matrice: array multidimensionale a 2 dimensioni



Matrici: excursus matematico

- In matematica, una matrice è uno schieramento rettangolare di oggetti (di solito numeri), organizzati in righe (orizzontali) e colonne (verticali)
- In generale, una matrice ha m righe e n colonne, con m e n interi positivi fissati
- Spesso una matrice è descritta indicando con a_{ij} il generico elemento posizionato alla riga i -esima e alla colonna j -esima
- I vettori si possono considerare matrici con una sola riga o una sola colonna (detti più precisamente vettore riga e vettore colonna)

$$\begin{bmatrix} 1 & 2 & 3 \\ 1 & -2 & 0 \\ 4.5 & 0 & 2 \\ 6 & 1 & 5 \end{bmatrix} \quad \begin{bmatrix} 7 \\ 0 \\ \pi \end{bmatrix} \quad \left[3 \quad \frac{7}{2} \quad -9 \right]$$

Matrici: dichiarazione e accesso

- La dichiarazione prevede l'indicazione esplicita del numero di elementi per ogni dimensione.
- Ipotizziamo di lavorare con un array a due dimensioni (il principio è il medesimo per un numero di dimensioni arbitrarie):

```
/* Dichiarazione di una matrice di interi  
   di dimensione N_RIGHE*N_COLONNE */  
int matrix[N_RIGHE][N_COLONNE];
```

- L'accesso avviene mediante due indici di riga e colonna esattamente come nel caso dei vettori.
- Valgono tutte le considerazioni fatte per i vettori, con l'unica eccezione legata al controllo sugli indici e alle conseguenze di una cattiva indicizzazione.

```
/* Stampa il contenuto della locazione  
   (3, 5) della matrice. */  
printf("%d", matrix[3][5]);
```

Array multidimensionali

- La memoria del calcolatore è lineare: è organizzata un insieme monodimensionale di celle identificate da un indirizzo sequenziale crescente
- Al momento della dichiarazione di un array, il calcolatore riserva un insieme contiguo di celle per contenere il vettore
 - Quindi: per un generico array `a`, la cella `a[2]` in memoria sarà adiacente alla cella `a[3]`, nella posizione immediatamente successiva
- Gli array multidimensionali vengono memorizzati come un unico array monodimensionale:
 - il primo elemento di ogni riga segue l'ultimo elemento della riga precedente
 - la possibilità di accedere con più indici rende trasparente questo aspetto al programmatore (zucchero sintattico)
- A prescindere dalla sua rappresentazione in memoria, una matrice può essere pensata come ad una tabella formata da righe e colonne:

```
/* Esempio di matrice e sua  
   rappresentazione in memoria */  
int matriceVoti[4][3];
```

Organizzazione in memoria

```
/* Esempio di matrice e sua  
   rappresentazione in memoria */  
int matriceVoti[4][3];
```

Rappresentazione NxM

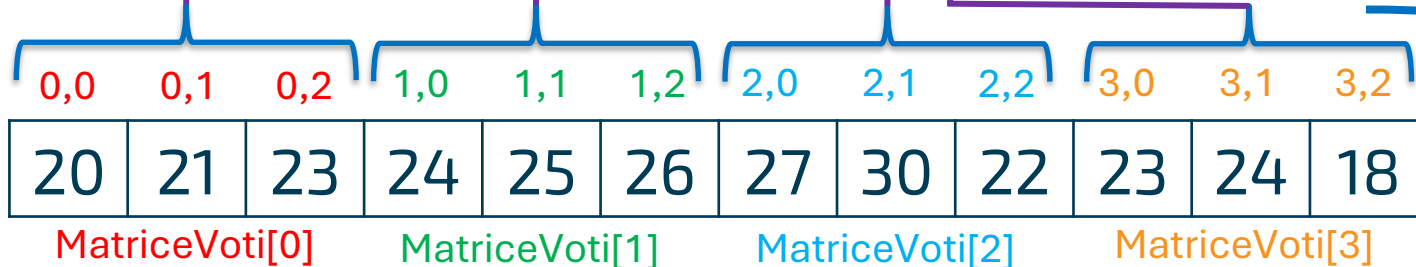
20	21	23
24	25	26
27	30	22
23	24	18

Riga 0

Riga 1

Riga 2

Riga 3



Memoria
del
calcolatore

Matrici: inizializzazione

Come per gli array monodimensionali, due modalità:

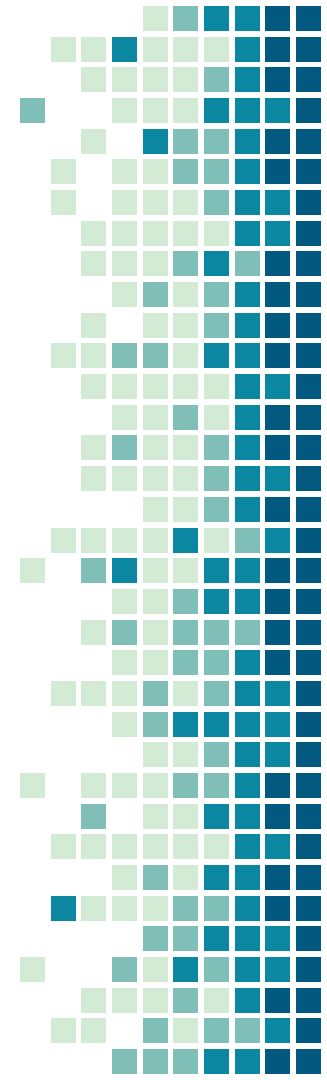
1. tramite costrutto iterativo (mediante doppio indice);

```
#define RIGHE 5
#define COLONNE 6

// Esempio di inizializzazione con costrutto iterativo
int mat[RIGHE][COLONNE];

...

for (r=0; r<RIGHE; r++)
{
    for (c=0; c<COLONNE; c++)
    {
        mat[r][c] = r + c;
    }
}
```



Matrici: inizializzazione

Come per gli array monodimensionali, due modalità:

2. in fase di dichiarazione (con alcune particolarità, dovute alla memoria lineare del calcolatore):

- La matrice è un array di array: ogni elemento riga è quindi a sua volta un vettore ed inizializzabile di conseguenza:

```
int a[2][3] = {{1,2,3},{4,5,6}};
```

- Allo stesso modo, è inizializzabile come array singolo, inserendo gli elementi in ordine di riga:

```
int a[2][3] = {1,2,3,4,5,6};
```

- Come per gli array, gli elementi non inizializzati vengono posti a zero:

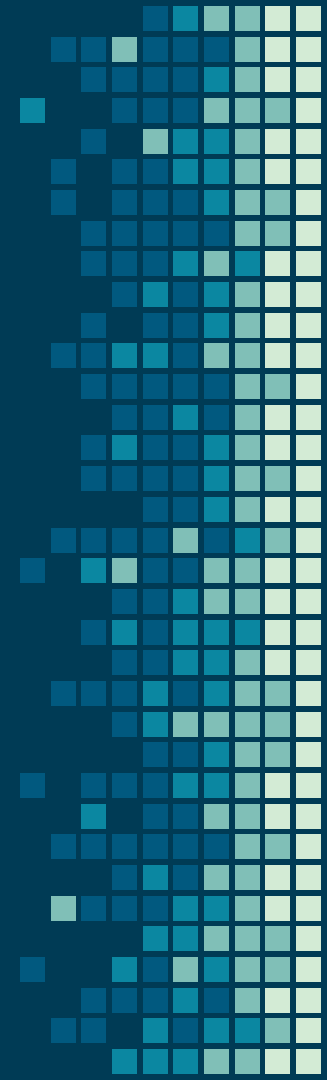
```
int a[2][3] = {{1,2},{4}};
```

- Quando ci sono abbastanza valori di inizializzazione per definire la dimensione della matrice, la prima dimensione può essere omessa e lasciata determinare dal compilatore:

```
int a[][3] = {{1,2},{4}};
```

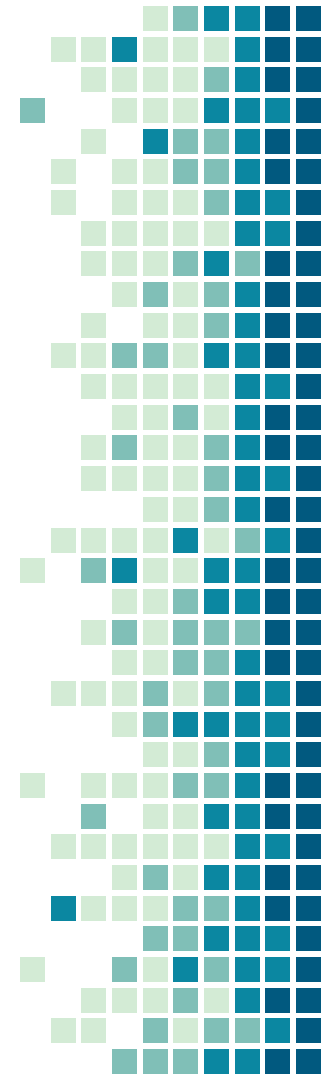
FINE!

Domande?



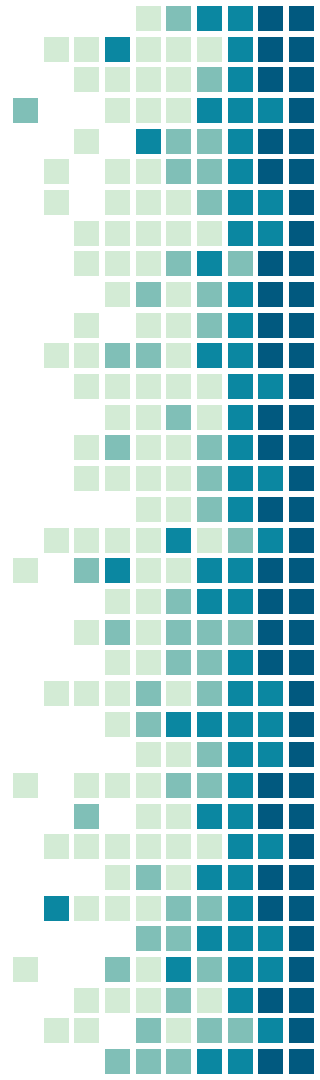
Autovalutazione

1. Come viene memorizzato in memoria un vettore bidimensionale con più di una riga?
2. Che differenza c'è tra array monodimensionali e bidimensionali? Quale coppia di indici rappresenta la prima cella di un vettore bidimensionale?
3. Con quali indici accedo alla cella della seconda colonna e terza riga?
4. Cosa succede se inizializzo una matrice senza specificarne la dimensione?
5. Posso copiare una matrice in un'altra utilizzando l'operatore di assegnamento?
6. Di che tipo può essere una matrice (o vettore bidimensionale)?
7. Quale segnaposto devo utilizzare per stampare il contenuto della cella di un vettore bidimensionale?
8. Con lo standard C90 posso dichiarare una matrice senza specificarne la dimensione? Se sì, come?



Esercizi

1. Scrivere un programma che stampi il contenuto di una matrice di interi, generati in maniera casuale nell'intervallo $[0, 100]$.
2. Scrivere un programma in cui si dichiara una matrice $N \times M$, popolata con valori casuali a piacere. Stampare la matrice e, successivamente, la somma di tutti i valori contenuti.
3. Scrivere un programma che stampi una matrice di float casuali dal primo all'ultimo elemento e successivamente dall'ultimo al primo.
Esempio: la matrice 3×3 $\{\{1,2,3\}, \{4,5,6\}, \{7,8,9\}\}$ sarà stampata come $\{\{9,8,7\}, \{6,5,4\}, \{3,2,1\}\}$
4. Generare una matrice identità (1 sulla diagonale principale, i restanti valori a 0) utilizzando un solo ciclo for.
5. Scrivere un programma in cui si genera casualmente una matrice $N \times M$ a piacere. Dichiarare un array R di N elementi e un array C di M elementi. Il programma deve salvare in $R[i]$ la somma degli elementi della riga i-esima della matrice, e in $C[j]$ la somma degli elementi della colonna j-esima della matrice.
6. Scrivere un programma in cui si dichiarano due matrici A e B, inizializzate a piacere. Dichiarare anche una terza matrice C che contenga la somma, elemento per elemento, delle altre due matrici.



Esercizi

7. Scrivere un programma in cui si dichiarano due matrici A e B. Dopo aver inizializzato A a piacere, fare in modo che la matrice B sia la matrice trasposta di A. Stampare A e B.
NB: la matrice trasposta è una matrice che ha, per righe, le colonne della matrice di partenza e, per colonne, le righe della matrice di partenza.

Esempio:

1 2 3
A = 4 5 6
7 8 9

1 4 7
B = 2 5 8
3 6 9

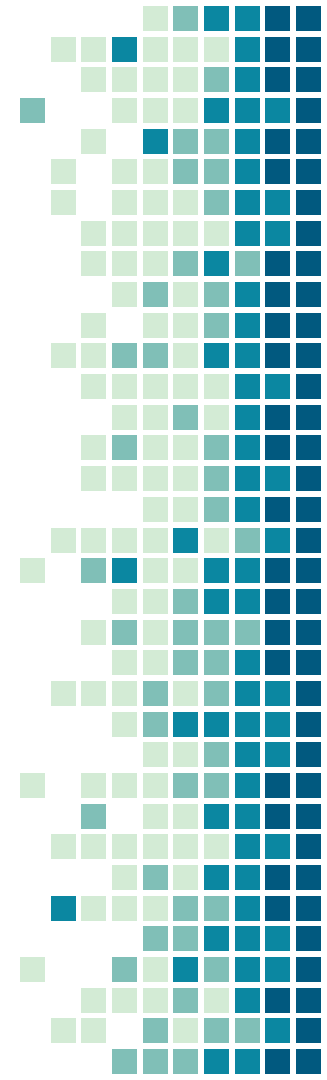
8. Scrivere un programma che definisca una matrice $N \times N$ di lato massimo 10, chieda all'utente di indicare la dimensione effettiva del lato (≤ 10) e la riempia di valori "a spirale" con valori crescenti da 0 a ($N^2 - 1$) iniziando dall'angolo superiore sinistro. Infine la visualizzi. Esempio:

0 1 2 3 4
15 16 17 18 5
14 23 24 19 6
13 22 21 20 7
12 11 10 9 8

Esercizi

9. Scrivere un programma in cui si dichiara una matrice P di dimensione $N \times N$ che rappresenta il tabellone di un campionato (ogni incrocio riga-colonna rappresenta una partita). P deve essere inizializzata in modo casuale, e l'elemento $P[i][j]$ può avere un valore che indica il pareggio, la vittoria della squadra di casa (squadra i -esima) o un valore che indica la vittoria della squadra in trasferta. Definire, quindi, un vettore C di dimensione N che contenga, per ciascuna squadra, il punteggio finale totalizzato (considerando 3 punti per la vittoria, 1 per il pareggio, 0 per le sconfitte). Indicare, in output, quale delle N squadre ha vinto il campionato.
10. Scrivere un programma per la rappresentazione del triangolo di Floyd. Il triangolo di Floyd è un triangolo rettangolo che contiene numeri naturali, definito riempiendo le righe del triangolo con numeri consecutivi e partendo da 1 nell'angolo in alto a sinistra. Costruire il triangolo di Floyd usando una matrice $N \times N$ e chiedendo all'utente un numero $n < N$ per riempire la matrice. Visualizzare, in output, il triangolo di Floyd.
Esempio: con $N = 5$, il triangolo di Floyd è il seguente:

```
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
```



Esercizi

11. Scrivere un programma che riempia una matrice, di dimensione a piacere, chiedendo all'utente di inserire gli elementi, ma inserendo nella matrice solo gli elementi pari. Il programma termina quando la matrice è piena.
12. Scrivere un programma che chieda all'utente di inserire i valori in una matrice di dimensione a piacere. Una volta terminato l'inserimento, il programma copia gli elementi dispari in una seconda matrice, di dimensione uguale alla prima, senza lasciare elementi contigui vuoti se non a "fondo matrice". Gli elementi in fondo devono essere messi a 0.

